Laboratórios de Análise de Discurso Político

Francisco Brandão Lucas M Pavelski

2026-06-03

Table of contents

Introdução **5**
Objetivos do curso . . . 6
Motivação . . . 6
Bibliografia . . . 6
Programa detalhado . . . 7
Aula 1 - 06/03/2026: . . . 7
Aula 2 - 20/03/2026: . . . 7
Aula 3 - 10/04/2026: . . . 7
Aula 4 - 15/05/2026: . . . 8
Aula 5 - 24/04/2026: . . . 8
Aula 6 - 22/05/2026: . . . 8
Aula 7 - 29/05/2026: . . . 8
Aula 8 - 26/06/2026: . . . 8
Avaliação . . . 8

1 Linguagem R **9**
1.1 Programando scripts em R: . . . 9
1.2 Ambientes para programar em R . . . 10
1.2.1 Rstudio . . . 11
1.2.2 Google Colab . . . 11
1.2.3 Console R . . . 11
1.2.4 Scripts R . . . 11
1.2.5 Notebooks R . . . 12
1.3 Comandos básicos . . . 13
1.3.1 Operadores aritméticos . . . 13
1.3.2 Operadores relacionais e lógicos . . . 13
1.3.3 Variáveis . . . 15
1.3.4 Condicionais . . . 18
1.3.5 Funções . . . 19
1.3.6 Vetores . . . 22
1.3.7 Data frames . . . 25
1.3.8 Bibliotecas e pacotes . . . 27
1.3.9 Strings (cadeias de caracteres) . . . 29
1.4 Exemplo: Análise de Sentimentos com R . . . 30
1.5 Para saber mais . . . 33

1.6 Atividade prática 33

2 Palavras, morfemas e tokens **34**
2.1 Obtendo dados textuais 34
2.1.1 Exemplo: Obtendo Discursos da Câmara dos Deputados 35
2.1.2 Criando objeto corpus 42
2.2 Palavras, morfemas, lemas e stems 45
2.2.1 Obtendo informações morfológicas de palavras 47
2.2.2 Stemming 48
2.2.3 Caracteres 49
2.2.4 Tokenização 49
2.2.5 Byte Pair Encoding (BPE) 52
2.2.6 Stopwords 53
2.3 Exemplo: Palavras mais Utilizadas em Discursos 54
2.4 Para saber mais 61
2.5 Atividade prática 62

Glossário **63**
Introdução 63
Aula 1 63

Referências **64**

Appendices **65**

Expressões regulares **65**

Introdução

Câmara dos Deputados
Centro de Formação, Treinamento e Aperfeiçoamento
Programa de Pós-Graduação

MESTRADO PROFISSIONAL EM PODER LEGISLATIVO
PLANO DE CURSO DE DISCIPLINA

DISCIPLINA: **ANÁLISE DO DISCURSO POLÍTICO**		
Período: **1º semestre 2026**	**Carga horária total:** **30 h/a**	**Código:** **MEST.7.09.17**

PROFESSORES	E-mail
FRANCISCO BRANDÃO, Dr.	francisco.brandao@camara.leg.br
LUCAS MARCONDES PAVELSKI, Dr.	lucas.pavelski@camara.leg.br

CURRÍCULOS RESUMIDOS

FRANCISCO BRANDÃO, Dr.
Grupo de Pesquisa e Extensão (GPE): Ciência de Dados Aplicada ao Estudo do Poder Legislativo: abordagem computacional e métodos de análise
Doutor em Ciência Política pela Universidade da Califórnia, Santa Barbara (2018), nos campos de Política Comparada, Comunicação Política e Tecnologia da Informação e Sociedade. Possui graduação em Jornalismo pela Universidade de São Paulo (2000) e mestrado em Ciência Política pela Universidade de Brasília (2008). Entre 2022 e 2023, foi pesquisador associado da Universidade de Loughborough, no Reino Unido, no projeto PANCOPOP – Comunicação Populista em Tempos de Pandemia. Jornalista na Diretoria de Comunicação da Câmara dos Deputados.

Currículo Lattes: http://lattes.cnpq.br/9403525530814720

LUCAS MARCONDES PAVELSKI, Dr.

Doutor e Mestre em Engenharia Elétrica e Informática Industrial pela Universidade Tecnológica Federal do Paraná (2021) com ênfase em sistemas inteligentes. Possui graduação em Ciência da Computação pela Universidade Estadual do Centro-Oeste do Paraná (2012). Atualmente é Analista de TI na Câmara dos Deputados, atuando na Seção de Soluções para a Área Política, com experiência em Ciência de Dados e Engenharia de Software.

Currículo Lattes: http://lattes.cnpq.br/9287626771427206

EMENTA DA DISCIPLINA

História, conceitos importantes, questões teóricas e método da Análise de Discurso (AD). Poder como controle, controle do discurso, estratégias discursivas, discurso e poder. Discurso como produção social. Formações discursivas e formações ideológicas. Relação entre estrutura e agência na produção discursiva. Análise argumentativa; discurso e argumentação. As instâncias midiáticas do discurso político. Relações entre discurso político e discurso midiático. Processos hermenêuticos envolvidos na AD. Narrativas, accounts, justificações, estratégias retóricas e uso de significantes vazios na política e nas instâncias jurídicas e midiáticas.

OBJETIVO GERAL DA DISCIPLINA

O aluno deverá compreender e aplicar empiricamente os conceitos básicos relacionados com as principais teorias de análise do discurso e seu instrumental metodológico.

OBJETIVOS ESPECÍFICOS DA DISCIPLINA

Objetivos do curso

Apresentar um ferramental tecnológico para auxiliar na análise de discursos políticos utilizando técnicas de Processamento de Linguagem Natural (PLN).

Motivação

- O volume de dados textuais disponíveis cresceu exponencialmente com a digitalização da informação e o advento das redes sociais.
- Pesquisa quantitativa complementa a pesquisa qualitativa, permitindo análises em larga escala.
- Ferramentas de PLN permitem extrair insights valiosos de grandes volumes de texto.
- Aplicações práticas em diversas áreas, como ciência política, sociologia, marketing e saúde pública.
- Desenvolvimento de habilidades técnicas em análise de dados textuais, programação e modelagem estatística.
- Foco prático do Mestrado Profissional em Poder Legislativo.

Bibliografia

JURAFSKY, D; MARTIN, J. H. Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models, 3rd edition, 2025.

Livro-texto com referencial teórico sobre processamento de linguagem natural, linguística computacional e análise de conteúdo (Jurafsky and Martin 2026).

Disponível em: https://web.stanford.edu/~jurafsky/slp3/

O'NEILL, L et al. Quantitative discourse analysis at Scale—AI, NLP and the transformer revolution. SoDa Laboratories Working Paper Series, v. 35, n. 11, p. 18-26, 2021.

Artigo apresentando NPL do estado-da-arte aplicada a estudos sociais (O'Neill et al. 2021) .

Disponível em: https://ideas.repec.org/p/ajr/sodwps/2021-12.html

SILGE, J.; ROBINSON, D. Text mining with R: a tidy approach. Bejing ; Boston: O'Reilly, 2017.

Livro-texto com referencial teórico e prático sobre mineração de textos com R e biblioteca tidytext (Silge and Robinson 2017).

Disponível em: https://www.tidytextmining.com/

> WICKHAM, Hadley et al. R for data science. Sebastopol: O'Reilly, 2017.

Livro-texto com referencial prático sobre ciência de dados com R (Wickham 2023).

Disponível em: https://r4ds.hadley.nz/ (inglês) e https://pt.r4ds.hadley.nz/ (tradução em português).

> CASELI, Helena de Medeiros; NUNES, Maria das Graças Volpe. Processamento de linguagem natural: conceitos, técnicas e aplicações em português. 2024.

Livro-texto com referencial teórico sobre processamento de linguagem natural em português (Caseli and Nunes 2024).

Disponível em: https://brasileiraspln.com/livro-pln/2a-edicao/

Programa detalhado

Aula 1 - 06/03/2026:

- Aplicações de PLN em análise de discurso político
- Linguagem R
- Operações básicas com textos em R

Aula 2 - 20/03/2026:

- Obtenção de dados textuais
- Dados do legislativo
- Morfemas, lemas e stemming
- Tokens e stopwords

Aula 3 - 10/04/2026:

- n-gramas
- Collocations
- TF-IDF
- Análise de tópicos

Aula 4 - 15/05/2026:

- Análise sintática
- Embeddings
- Similaridade de Documentos

Aula 5 - 24/04/2026:

- Classificação de textos
- Modelos de linguagem

Aula 6 - 22/05/2026:

- Análise de sentimentos
- Reconhecimento de entidades nomeadas

Aula 7 - 29/05/2026:

- Modelos discursivos
- Retrieval Augmented Generation (RAG)

Aula 8 - 26/06/2026:

- Apresentações finais

Avaliação

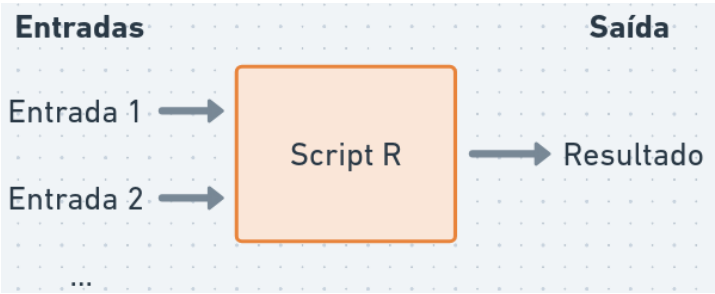
A avaliação será feita com base na participação nas aulas e na apresentação final de um projeto prático de análise de discurso político utilizando técnicas de processamento de linguagem natural.

1 Linguagem R

> **Nesta aula**
>
> - Como utilizar a linguagem R
> - Comandos básicos
> - Utilizando bibliotecas
> - Funções para processamento de texto
>
> Link para Google Colab: https://drive.google.com/file/d/1LalxshzmyMmTyfuIaFbp7RYcoP5Qnvuo/view?usp=sharing

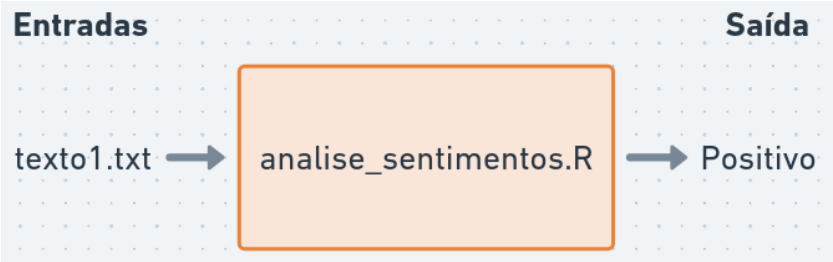
- **R** é uma linguagem de programação e ambiente de software para computação estatística e gráficos
- Criada por Ross Ihaka e Robert Gentleman em 1993
- Muito utilizada em ciência de dados, estatística e análise de texto
- Possui uma grande comunidade de usuários e desenvolvedores, com muitas bibliotecas para análise de textos

1.1 Programando scripts em R:



Script R: sequência de comandos R que podem ser executados para realizar uma tarefa específica (ex: análise de sentimentos)

Em PLN:



`analise_sentimentos.R` é um arquivo contendo instruções para:

- Ler arquivos de texto
- Processar textos (tokenização, remoção de stopwords, stemming)
- Analisar sentimentos (encontrar palavras positivas e negativas)
- Gerar saída (relatório, gráfico, tabela)

O **interpretador R (Rscript)** é um programa que executa os comandos do script e produz a saída desejada. Por exemplo, pelo terminal de comandos:

```
{$ Rscript analise_sentimentos.R texto1.txt} > "Positivo"
```

1.2 Ambientes para programar em R

O interpretador R está disponível para download em https://cran-r.c3sl.ufpr.br/

```r
## (código utilizado apenas para exibir páginas no Google Colab)
if (!requireNamespace("htmltools", quietly = TRUE)) {
  install.packages("htmltools")
}
htmltools::tags$iframe(src = "https://cran-r.c3sl.ufpr.br/", width = "100%", height = "315")
```

- No Windows, é necessário também instalar o RTools, disponível em https://cran.r-project.org/bin/windows/Rtools/, que inclui ferramentas para auxiliar a instalação de pacotes do R.

Podemos escrever comandos R em diversos ambientes:

1.2.1 Rstudio

- Ambiente de programação integrado (IDE) instalada localmente (Windows, Mac, Linux).
- Possui diversas funcionalidades para facilitar a escrita e execução de código R, como editor de scripts, console interativo, visualização de gráficos e gerenciamento de pacotes.

Disponível em: https://posit.co/downloads/.

Uma alternativa é o IDE Positron, disponível em: https://positron.posit.co/download.html.

1.2.2 Google Colab

- Ambiente online para programação em Python, mas também suporta R com algumas configurações adicionais (selecione "R" como linguagem de programação para criar um notebook R).
- Permite compartilhar notebooks interativos com código R e resultados.

Disponível em: https://colab.research.google.com/.

Outra alternativa é a posit.cloud, disponível em: https://posit.cloud.

1.2.3 Console R

- Interface de linha de comando para executar comandos R diretamente no terminal.
- Um comando é executado por vez (digite o comando e aperte Enter), e o resultado é exibido imediatamente.
- É um ambiente menos amigável para desenvolver scripts longos, mas útil para testes rápidos e para o aprendizado da linguagem.

Acessando o console R:

- No terminal, digite `R` para iniciar o console R e `q()` para sair.
- No RStudio, o console R está disponível na parte inferior da interface.
- No Google Colab, clique em "Terminal" e digite "R" para iniciar o console R.

1.2.4 Scripts R

- Arquivo de texto com extensão `.R` que contém uma sequência de comandos R.
- Permite organizar e reutilizar código, facilitando a execução de tarefas complexas.

Como escrever um script R:

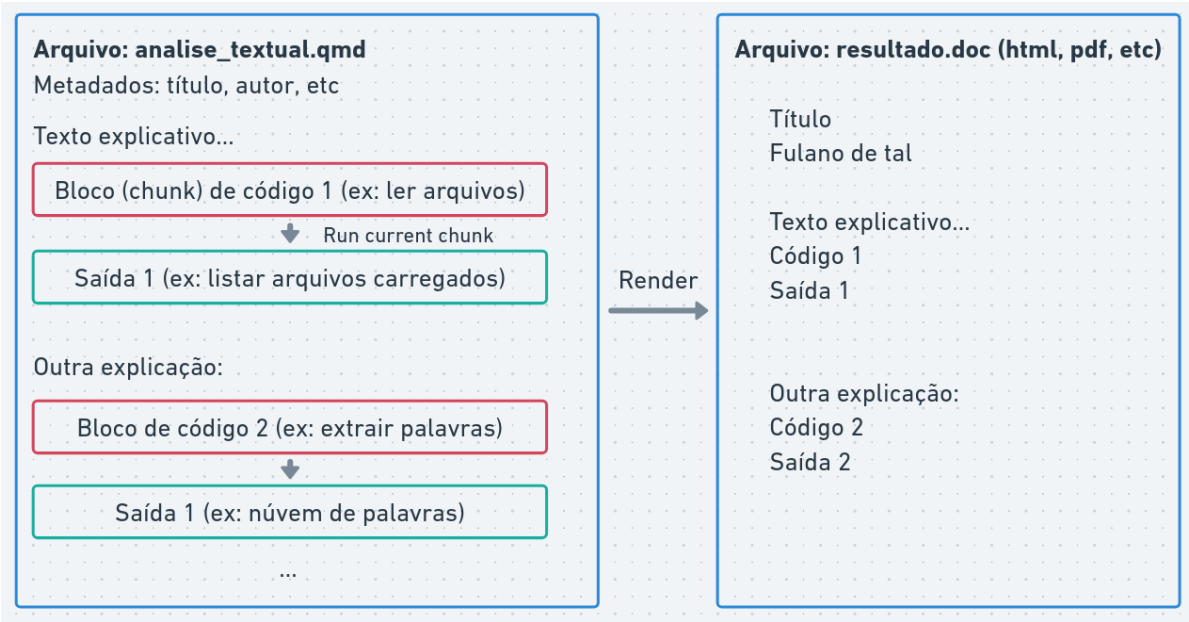
- No RStudio, crie um novo script em: `File > New File > R Script`. Clique em "Source" para executar o script inteiro ou selecione linhas específicas e clique em "Run" para executar apenas essas linhas.

1.2.5 Notebooks R

- Notebooks são documentos que combinam texto explicativo, código R e resultados (gráficos, tabelas) em um formato interativo.
- Permitem criar relatórios dinâmicos e interativos, facilitando a comunicação dos resultados.

Para criar e executar notebooks R, você pode usar o Rstudio ou o Google Colab:

- No Rstudio, crie um novo notebook: File > New File > Quarto Document.
- No Google Colab, clique em "File" > "New Notebook" e selecione "R" como linguagem de programação.



Notebooks possuem dois tipos de células:

- células de texto para escrever explicações e comentários
- células de código para escrever e executar códigos em R (o botão "play" executa cada célula individualmente).

1.3 Comandos básicos

- Executar cálculos
- Operadores relacionais e lógicos
- Variáveis
- Listas e vetores
- Funções
- Strings (cadeias de caracteres)

1.3.1 Operadores aritméticos

R pode ser utilizado para cálculos básicos, por exemplo:

```
print(4 + 3)
```

```
[1] 7
```

- `print(*)` é uma função que exibe o resultado de uma expressão na tela.

```
print(5 * 2 + 3)
```

```
[1] 13
```

```
print((6 - 1) / 2)
```

```
[1] 2.5
```

Qualquer código após `#` é considerado um comentário e não é executado:

```
print(8 / 3)     # Isto é um comentário, não será executado
```

```
[1] 2.666667
```

1.3.2 Operadores relacionais e lógicos

Operadores relacionais são utilizados para comparar valores.

Igualdade:

```r
print(5 == 5)
```

```
[1] TRUE
```

Diferença:

```r
print(1 != 2)
```

```
[1] TRUE
```

Maior que:

```r
print(4 > 3)
```

```
[1] TRUE
```

Menor que:

```r
print(2 < 5)
```

```
[1] TRUE
```

Maior ou igual a:

```r
print(3 >= 3)
```

```
[1] TRUE
```

- Expressões lógicas: retornam verdadeiro (`TRUE`) ou falso (`FALSE`)

Conjunção "e":

```r
print(4 > 3 && 8 > 2)
```

```
[1] TRUE
```

- `&&` é o operador lógico "e" (conjunção) que retorna `TRUE` se ambas as expressões forem verdadeiras, caso contrário retorna `FALSE`:

- `TRUE  && TRUE` retorna `TRUE`
- `TRUE  && FALSE` retorna `FALSE`
- `FALSE && TRUE` retorna `FALSE`
- `FALSE && FALSE` retorna `FALSE`

Disjunção "ou":

```
print(4 > 3 || 2 > 8)
```

```
[1] TRUE
```

- `||` é o operador lógico "ou" (disjunção), que retorna `TRUE` se pelo menos uma das expressões for verdadeira. Caso contrário, retorna `FALSE`.
- `TRUE  || TRUE` retorna `TRUE`
- `TRUE  || FALSE` retorna `TRUE`
- `FALSE || TRUE` retorna `TRUE`
- `FALSE || FALSE` retorna `FALSE`

Negação "não"

```
print(!(5 > 3))
```

```
[1] FALSE
```

- `!` é o operador lógico "não" (negação) que inverte o valor lógico de uma expressão. Se a expressão for `TRUE`, retorna `FALSE`, e se for `FALSE`, retorna `TRUE`.
- `!(TRUE)` retorna `FALSE`
- `!(FALSE)` retorna `TRUE`

```
print((TRUE && FALSE) || TRUE)
```

```
[1] TRUE
```

1.3.3 Variáveis

Variáveis são utilizadas para guardar valores na memória do computador:

```r
a <- 5

print(a)
```

```
[1] 5
```

Utilizamos o operador `<-` para atribuir valores a variáveis.

O valor armazenado pode ser utilizado em outras expressões:

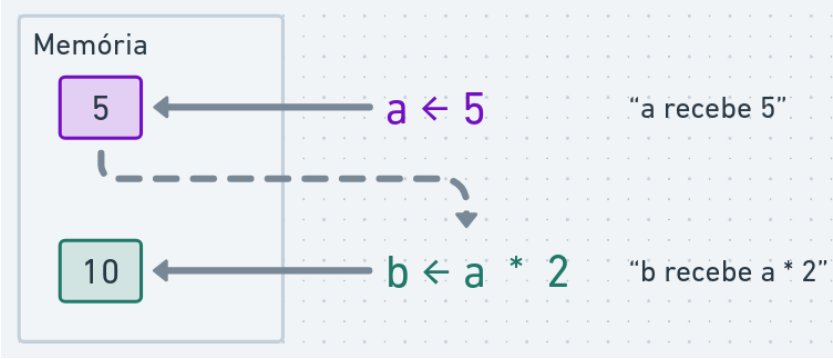
```r
print(a * 2 + 3)
```

```
[1] 13
```

Ou podemos criar novas variáveis a partir de outras:

```r
b <- a * 2

print(b)
```

```
[1] 10
```



O valor armazenado na variável pode ser alterado:

```r
a <- 10

print(a)
```

```
[1] 10
```

Variáveis podem armazenar diferentes tipos:

```r
p <- 1 == 1
q <- 4 < 3

print(p || q)
```

```
[1] TRUE
```

O nome de uma variável deve começar com uma letra e pode conter letras, números (exceto no começo), underscores (_) e pontos:

```r
isto_é_uma_variável <- 10
aluno_aprovado_1 <- isto_é_uma_variável == 10

print(isto_é_uma_variável)
```

```
[1] 10
```

```r
print(aluno_aprovado_1)
```

```
[1] TRUE
```

Exemplo de cálculo do índice de massa corporal (IMC):

```r
peso <- 70 # peso em kg
altura <- 1.75 # altura em metros

imc <- peso / (altura * altura)

imc_saudavel <- imc >= 18.5 && imc <= 24

print(imc)
```

```
[1] 22.85714
```

```r
print(imc_saudavel)
```

```
[1] TRUE
```

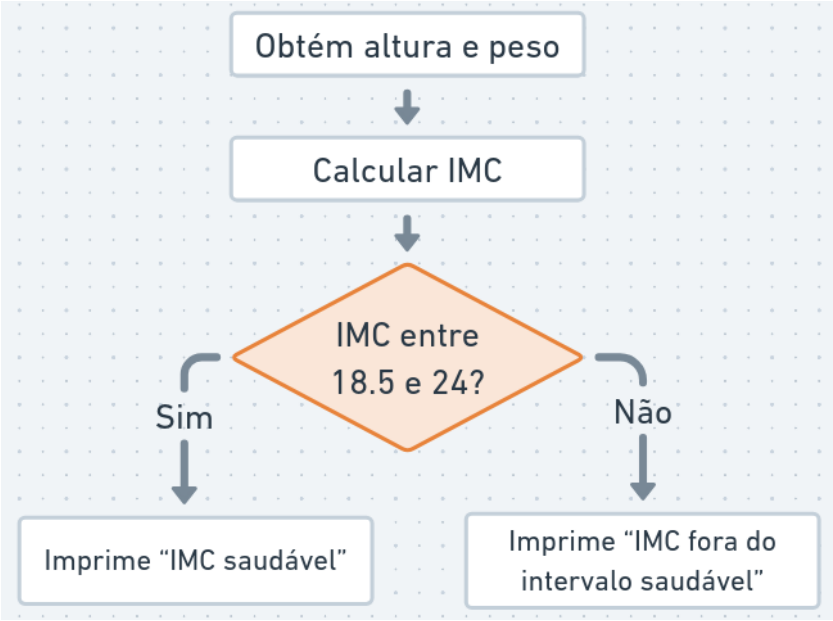
1.3.4 Condicionais

O comando `if` é utilizado para executar um bloco de código apenas se uma condição for verdadeira:

```r
x <- 10

if (x > 5) {
  print("x é maior que 5")
}
```

```
[1] "x é maior que 5"
```

O comando `else` pode ser utilizado para executar um bloco de código alternativo caso a condição seja falsa:



```r
peso <- 70 # peso em kg
altura <- 1.75 # altura em metros

imc <- peso / (altura * altura)

print(imc)
```

```
[1] 22.85714
```

```r
imc_saudavel <- imc >= 18.5 && imc <= 24

if (imc_saudavel) {
  print("IMC saudável")
} else {
  print("IMC fora do intervalo saudável")
}
```

```
[1] "IMC saudável"
```

1.3.5 Funções

Uma **função** é um bloco de código que possui um nome e realiza uma tarefa específica, geralmente para operações comuns que se repetem.

Por exemplo, `print` é a função que usamos para exibir resultados na tela.

R possui várias funções para estatística e análise de dados. Alguns exemplos são:

Valor absoluto:

```r
print(abs(-5))
```

```
[1] 5
```

Raiz quadrada:

```r
raiz <- sqrt(16)

print(raiz)
```

```
[1] 4
```

Logaritmo:

```r
log(100, base = 10)
```

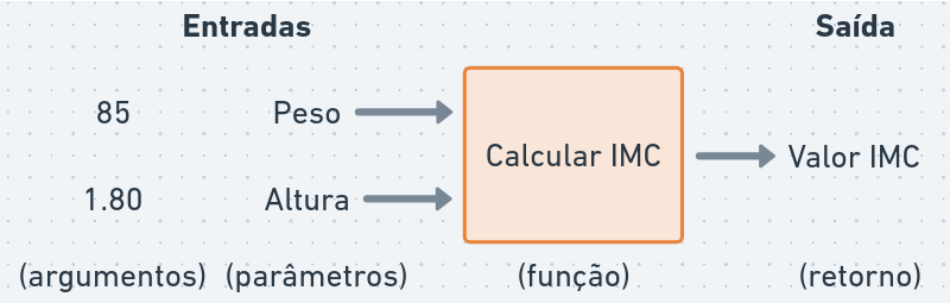
```
[1] 2
```

- O número 100 é o primeiro argumento da função.
- O número 10 é o segundo argumento, nomeado como base. Por padrão, a base é o número de Euler (exp(1)), calculando o logaritmo natural.
- Para obter ajuda sobre uma função, execute `help(log)` ou `?log`.

Também é possível definir nossas próprias funções:

```r
calcular_imc <- function(peso, altura) {
  altura_quadrado <- altura * altura
  return(peso / altura_quadrado)
}

imc <- calcular_imc(85, 1.80)
print(imc)
```

```
[1] 26.23457
```



Observe a sintaxe para declarar uma função:

```
nome_da_funcao <- function(argumentos) {
  # corpo da função
}
```

A expressão `return(...)` é utilizada para indicar o valor que a função deve retornar. Se `return()` não for especificado, a função retornará o resultado da última expressão avaliada em seu corpo.

Note que qualquer variável criada dentro de uma função é uma variável **local**, ou seja, ela não pode ser acessada fora do escopo da função:

```
print(altura_quadrado)
```

```
Error in print(altura_quadrado): objeto 'altura_quadrado' não encontrado
```

Após definir uma função, podemos utilizá-la em qualquer parte do código:

```
teste_imc <- function(peso, altura) {
  imc <- calcular_imc(peso, altura)
  imc_saudavel <- imc >= 18.5 && imc <= 24
  return(imc_saudavel)
}

print(calcular_imc(80, 1.75) < calcular_imc(70, 1.65))
```

```
[1] FALSE
```

Em certos casos, utilizamos o resultado de uma função como argumento de outra função:

```
converter_para_celsius <- function(fahrenheit) {
  celsius <- (fahrenheit - 32) * 5 / 9
  return(celsius)
}

esta_congelando <- function(celsius) {
  congelando <- celsius <= 0
  return(congelando)
}

temperatura_inverno <- 25 # Graus Fahrenheit (aprox. -3.8 °C)

resultado <- esta_congelando(converter_para_celsius(temperatura_inverno))

print(resultado)
```

```
[1] TRUE
```

Em R, podemos utilizar o *operador pipe* (`|>`) para encadear funções de forma mais legível:

```r
temperatura_inverno <- 25 # Graus Fahrenheit (aprox. -3.8 °C)

resultado <- temperatura_inverno |>
  converter_para_celsius() |>
  esta_congelando()

print(resultado)
```

```
[1] TRUE
```

- No exemplo, a expressão `temperatura_inverno` é passada como primeiro argumento para a função `converter_para_celsius()`, e o resultado dessa função é então passado como primeiro argumento para a função `esta_congelando()`.

O operador pipe (`|>`) torna o código mais legível, permitindo que as funções sejam encadeadas de maneira clara e fluida.

1.3.6 Vetores

Vetores são estruturas que armazenam uma sequência de elementos do mesmo tipo:

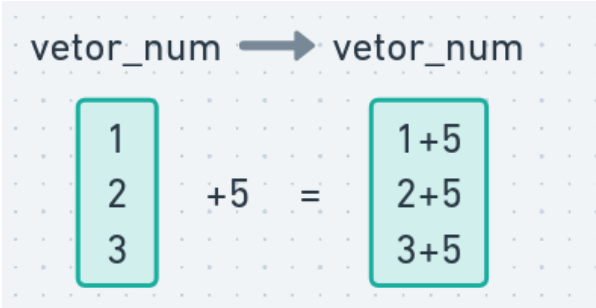
```r
vetor_num <- c(1, 2, 3, 4, 5, 6, 7)

print(vetor_num)
```

```
[1] 1 2 3 4 5 6 7
```

Operações aritméticas podem ser aplicadas a todos os elementos de um vetor simultaneamente:

```r
vetor_num <- c(1, 2, 3)
vetor_num <- vetor_num + 5

print(vetor_num)
```

```
[1] 6 7 8
```



É possível criar sequências numéricas de forma simples:

```
vetor_seq <- 1:10

print(vetor_seq)
```

```
 [1]  1  2  3  4  5  6  7  8  9 10
```

Algumas funções operam sobre vetores:

```
print(sum(1:100))
```

```
[1] 5050
```

```
print(mean(1:100))
```

```
[1] 50.5
```

```
print(sd(1:100))
```

```
[1] 29.01149
```

Para acessar um elemento específico do vetor, utilizamos colchetes `[]`:

```
vetor_pares <- c(2, 4, 6, 8, 10)

print(vetor_pares[3])
```

```
[1] 6
```

Para alterar elementos do vetor também utilizamos colchetes:

```r
vetor_seq <- 1:10
vetor_seq[3] <- 999

print(vetor_seq)
```

```
 [1]   1   2 999   4   5   6   7   8   9  10
```

Podemos acessar múltiplos elementos do vetor:

```r
print(vetor_pares[c(1, 4)])
```

```
[1] 2 8
```

Também podemos filtrar elementos do vetor com expressões lógicas:

```r
pares_maior_5 <- vetor_pares > 5
elementos_maior_5 <- vetor_pares[pares_maior_5]

print(vetor_pares)
```

```
[1]  2  4  6  8 10
```

```r
print(pares_maior_5)
```

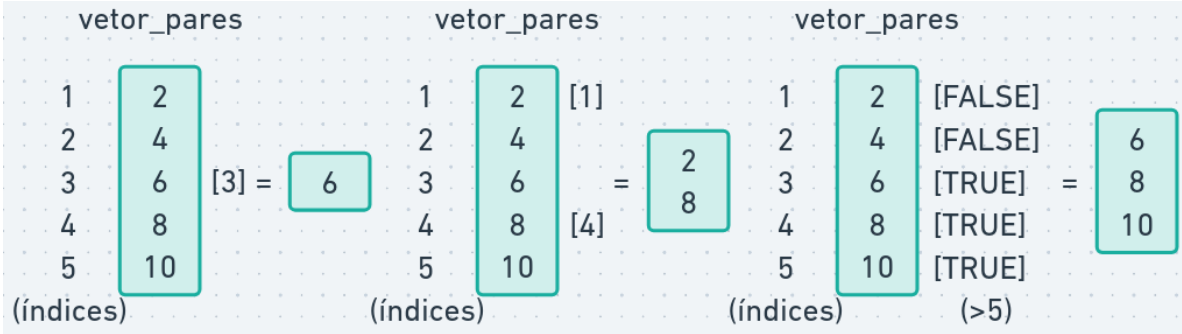
```
[1] FALSE FALSE  TRUE  TRUE  TRUE
```

```r
print(elementos_maior_5)
```

```
[1]  6  8 10
```

```r
print(vetor_pares[vetor_pares > 5])
```

```
[1]  6  8 10
```



O operador `%in%` é utilizado para verificar se certos elementos estão presentes em um vetor:

```
vetor_a <- c(2, 3, 10)

print(3 %in% vetor_a)
```

```
[1] TRUE
```

```
vetor_b <- c(3, 4, 5, 6, 7)

print(vetor_a %in% vetor_b)
```

```
[1] FALSE  TRUE FALSE
```

1.3.7 Data frames

R possui vários tipos de dados, os principais são:

- Numéricos: números inteiros ou decimais (ex: `5`, `3.14`)
- Lógicos: `TRUE` ou `FALSE`
- Strings: texto entre aspas (ex: `"Olá, mundo!"`)

Data frames são estruturas de dados tabulares, onde cada coluna é um vetor e cada linha representa uma observação.

```
df <- data.frame(
  nome = c("João", "Maria", "Pedro"),
  idade = c(30, 25, 28),
  turma = c("A", "B", "A")
)

print(df)
```

```
   nome idade turma
1  João    30     A
2 Maria    25     B
3 Pedro    28     A
```

Data frames são amplamente utilizados para análise de dados, permitindo manipular e analisar conjuntos de dados tabulares.

Essencialmente, um data frame é uma lista de vetores de mesmo comprimento, em que cada vetor representa uma coluna. Cada coluna pode conter um tipo diferente de dado (numérico, lógico, texto etc.).

Acessando colunas:

```r
df <- data.frame(
  nome = c("João", "Maria", "Pedro"),
  idade = c(30, 25, 28),
  turma = c("A", "B", "A")
)

print(df$idade)
```

```
[1] 30 25 28
```

```r
print(df[, "idade"])
```

```
[1] 30 25 28
```

Acessando linhas:

```r
print(df[1, ])
```

```
  nome idade turma
1 João    30     A
```

Acessando linha e coluna:

```r
print(df[1, "idade"])
```

```
[1] 30
```

Para selecionar alunos com mais de 25 anos:

```r
alunos_mais_velhos <- df[df$idade > 25, ]
print(alunos_mais_velhos)
```

```
   nome idade turma
1  João    30     A
3 Pedro    28     A
```

1.3.8 Bibliotecas e pacotes

*Bibliotecas** (ou pacotes) são conjuntos de funções, dados e documentação que estendem as funcionalidades do R. Geralmente são escritas por outros usuários e compartilhadas com a comunidade (ex: `tidyverse`).

Para instalar uma biblioteca, utilizamos o comando `install.packages()`.

```r
if (!"tidyverse" %in% installed.packages()) {
  install.packages("tidyverse")
}
```

Pesquisadores do mundo todo contribuem com bibliotecas no repositório oficial do R, https://cran-r.c3sl.ufpr.br/.

Para acessar as funções de uma biblioteca, é necessário carregá-la com `library(...)`:

```
-- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
v dplyr     1.1.4     v readr     2.1.5
v forcats   1.0.0     v stringr   1.5.1
v ggplot2   3.5.2     v tibble    3.3.0
v lubridate 1.9.4     v tidyr     1.3.1
v purrr     1.0.4
-- Conflicts ------------------------------------------ tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()    masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to becom
```

O tidyverse é uma coleção de pacotes para ciência de dados que compartilham uma filosofia de design.

Ele inclui pacotes para manipulação de dados (`dplyr`, `tidyr`), visualização (`ggplot2`) e muito mais.

Suas funções para manipulação de data frames (aqui chamados de tibbles), como `filter`, `select` e `mutate`, facilitam a análise de dados tabulares:

```
alunos <- tibble(
  nome = c("João", "Maria", "Pedro"),
  idade = c(30, 25, 28),
  nota_final = c(8.5, 9.0, 6.5),
  turma = c("A", "B", "A")
)

alunos_filtrados <- alunos |>
  filter(idade > 25) |>                  # filtrar os alunos com mais de 25 anos
  mutate(aprovado = nota_final >= 7) |>  # nova coluna "aprovado" com base na nota final
  select(nome, idade, aprovado)          # selecionar apenas as colunas nome e idade

print(alunos_filtrados)
```

```
# A tibble: 2 x 3
  nome  idade aprovado
  <chr> <dbl> <lgl>
1 João     30 TRUE
2 Pedro    28 FALSE
```

Calculando a média de idade dos alunos aprovados:

```
media_idade_aprovados <- alunos |>
  filter(nota_final >= 7) |>
  summarise(media_idade = mean(idade))

print(media_idade_aprovados)
```

```
# A tibble: 1 x 1
  media_idade
        <dbl>
1        27.5
```

Em resumo, vimos exemplos de funções:

- `filter()`: filtra linhas com base em condições lógicas
- `mutate()`: cria novas colunas ou modifica colunas existentes
- `select()`: seleciona colunas específicas

- `summarise()`: calcula estatísticas resumidas, como média, soma, etc

Existem várias outras funções para manipulação de data frames incluídas no `tidyverse`. Um resumo visual dessas funções está disponível em https://nyu-cdsc.github.io/learningr/assets/data-transformation.pdf.

1.3.9 Strings (cadeias de caracteres)

String é o tipo de dado utilizado para representar textos. Em R, strings são definidas com aspas simples ou duplas:

```
texto <- "Olá, mundo!"

print(texto)
```

```
[1] "Olá, mundo!"
```

A biblioteca `stringr` (parte do `tidyverse`) possui diversas funções para a manipulação de strings (todas começam com o prefixo `str_`):

```
a <- "olá"
b <- 'mundo'
a_b_concatenados <- str_c(a, b, sep = " ")

print(a_b_concatenados)
```

```
[1] "olá mundo"
```

- Concatenar significa unir duas ou mais strings. A função str_c do pacote stringr é usada para esta finalidade, e o argumento `sep` especifica o caractere separador a ser inserido entre as strings (neste caso, um espaço).

```
num_palavras <- str_length("quantos caracteres tem esta frase?")

print(num_palavras)
```

```
[1] 34
```

- A função `str_length` retorna o número de caracteres em uma string, incluindo espaços e pontuação.

```r
contem_texto <- str_detect("este texto contém palavras", "texto")

print(contem_texto)
```

```
[1] TRUE
```

- A função `str_detect` verifica se uma string contém um padrão de texto. Ela retorna TRUE se o padrão for encontrado e FALSE caso contrário.

As funções do pacote `stringr` são vetorizadas, o que significa que elas podem ser aplicadas diretamente a um vetor de textos, retornando um resultado para cada elemento.

```r
discursos <- c(
  "este texto contém palavras",
  "este outro texto não tem a palavra-chave",
  "a palavra-chave está presente aqui"
)
contem_texto <- str_detect(discursos, "palavra-chave")

print(contem_texto)
```

```
[1] FALSE  TRUE  TRUE
```

```r
print(discursos[contem_texto])
```

```
[1] "este outro texto não tem a palavra-chave"
[2] "a palavra-chave está presente aqui"
```

1.4 Exemplo: Análise de Sentimentos com R

```r
library(tidyverse)
```

Criando um vetor de textos:

```r
textos <- c(
  "Eu amo este produto, é incrível!",
  "Este serviço é péssimo, estou muito insatisfeito.",
  "O filme foi ótimo, adorei a história e os personagens.",
  "A comida estava horrível, não recomendo este restaurante.",
  "O atendimento foi excelente, muito atenciosos e rápidos.",
  "Não volto mais a este lugar."
)

print(textos)
```

```
[1] "Eu amo este produto, é incrível!"
[2] "Este serviço é péssimo, estou muito insatisfeito."
[3] "O filme foi ótimo, adorei a história e os personagens."
[4] "A comida estava horrível, não recomendo este restaurante."
[5] "O atendimento foi excelente, muito atenciosos e rápidos."
[6] "Não volto mais a este lugar."
```

Criando um data frame com os textos:

```r
df_textos <- tibble(texto = textos)

print(df_textos)
```

```
# A tibble: 6 x 1
  texto
  <chr>
1 Eu amo este produto, é incrível!
2 Este serviço é péssimo, estou muito insatisfeito.
3 O filme foi ótimo, adorei a história e os personagens.
4 A comida estava horrível, não recomendo este restaurante.
5 O atendimento foi excelente, muito atenciosos e rápidos.
6 Não volto mais a este lugar.
```

Criando um vetor de palavras positivas e negativas:

```r
palavras_positivas <- c("amo", "incrível", "ótimo",
                        "adorei", "excelente", "atenciosos", "rápidos")
palavras_negativas <- c("péssimo", "insatisfeito", "horrível",
                        "não recomendo", "ruim", "lento")
```

```r
print(palavras_positivas)
```

```
[1] "amo"        "incrível"   "ótimo"      "adorei"     "excelente"
[6] "atenciosos" "rápidos"
```

```r
print(palavras_negativas)
```

```
[1] "péssimo"       "insatisfeito"  "horrível"      "não recomendo"
[5] "ruim"          "lento"
```

Criando uma função para analisar o sentimento de um texto:

```r
analisar_sentimento <- function(palavras) {
  palavras_positivas_encontradas <- sum(palavras %in% palavras_positivas)
  palavras_negativas_encontradas <- sum(palavras %in% palavras_negativas)
  if (palavras_positivas_encontradas > palavras_negativas_encontradas) {
    return("Positivo")
  } else if (palavras_negativas_encontradas > palavras_positivas_encontradas) {
    return("Negativo")
  } else {
    return("Neutro")
  }
}

print(analisar_sentimento(c("este", "produto", "é", "incrível")))
```

```
[1] "Positivo"
```

Aplicando a função de análise de sentimento a cada texto:

```r
df_analise <- df_textos |>
  mutate(texto_min = str_to_lower(texto)) |>                        # converter para minúscu
  mutate(palavras = str_extract_all(texto_min, boundary("word"))) |> # dividir o texto em pal
  mutate(sentimento = map_chr(palavras, analisar_sentimento)) |>     # map_chr aplica a funçã
  select(texto, sentimento)                                          # selecionar apenas as c

print(df_analise)
```

```
# A tibble: 6 x 2
  texto                                                   sentimento
  <chr>                                                   <chr>
1 Eu amo este produto, é incrível!                        Positivo
2 Este serviço é péssimo, estou muito insatisfeito.       Negativo
3 O filme foi ótimo, adorei a história e os personagens.  Positivo
4 A comida estava horrível, não recomendo este restaurante. Negativo
5 O atendimento foi excelente, muito atenciosos e rápidos. Positivo
6 Não volto mais a este lugar.                            Neutro
```

```r
df_contagem <- df_analise |>
  group_by(sentimento) |>    # agrupar por sentimento
  summarise(contagem = n())  # contar o número de textos em cada categoria de sentimento

print(df_contagem)
```

```
# A tibble: 3 x 2
  sentimento contagem
  <chr>         <int>
1 Negativo          2
2 Neutro            1
3 Positivo          3
```

1.5 Para saber mais

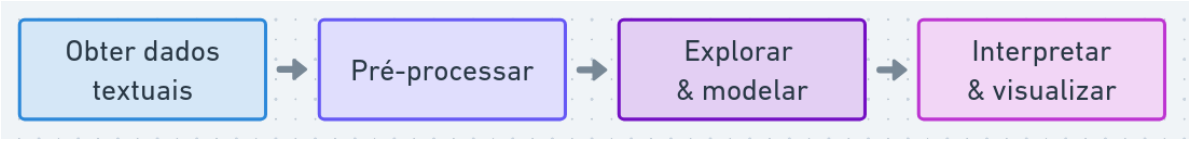
As três primeiras partes do livro *R for Data Science* (Wickham 2023) são uma ótima introdução à linguagem R e à manipulação de dados com a biblioteca `tidyverse`.

1.6 Atividade prática

1. Crie um vetor de textos contendo pelo menos 5 frases de discursos políticos.
2. Crie um data frame com esses textos.
3. Defina 3 vetores com palavras representando temas políticos (ex: economia, saúde, educação).
4. Crie uma função que analise um texto, identifique a qual tema ele mais se relaciona e retorne o tema predominante.
5. Aplique a função de análise de temas a cada texto e crie uma nova coluna no data frame indicando o tema predominante em cada texto.

2 Palavras, morfemas e tokens

2.1 Obtendo dados textuais



Quais são fontes textuais interessantes para estudos políticos?

Tipos de fontes de dados textuais:

- *Banco de dados*: dados já organizados e estruturados para análise.

  Muitas vezes inclúem metadados (informações adicionais sobre os textos, ex: dada, interlocutor, local etc).

  Como o custo de manutenção é alto, o acesso geralmente é restrito para a instituição.

- *APIs*: interfaces de programação que permitem acessar dados textuais de forma automatizada, como APIs de redes sociais (Twitter, Facebook) e dados governamentais (dados.gov.br)

  O formato dos dados varia para cada fonte (JSON ou XML), mas geralmente inclui metadados e o texto em si.

  O acesso pode ser fechado (chave de API) ou limitado (limite de requisições por dia).

- *Arquivos de texto*: dados em formato bruto, como arquivos de texto (TXT, CSV) e texto com formatação (Markdown, HTML, RTF)

  A leitura desses arquivos por programas é fácil, mas pode exigir pré-processamento para lidar com formatação e metadados.

- *Arquivos não estruturados*: dados em formatos não estruturados, como PDFs, documentos Word, planilhas, imagens e vídeos.

  PDFs e imagens e vídeos podem ser convertidos em texto utilizando técnicas de OCR (Optical Character Recognition) ou transcrição automática.

  O processamento pode ser custoso e gerar ruídos.

- *Web scraping*: extração de dados textuais de sites da internet utilizando ferramentas como `rvest` em R.

  O acesso é aberto, mas pode ser limitado por políticas de uso do site ou por medidas de segurança (CAPTCHA, bloqueio de IP).

  O formato dos dados varia para cada site e pode exigir pré-processamento para lidar com formatação e metadados. Alterações no site podem quebrar o processo de extração.

2.1.1 Exemplo: Obtendo Discursos da Câmara dos Deputados

A Câmara dos Deputados disponibiliza um banco de dados de discursos em formato XML, que pode ser acessado por meio de uma API.

A documentação da API com exemplos está disponível em: https://www2.camara.leg.br/transparencia/dados-abertos/dados-abertos-legislativo/webservices/sessoesreunioes-2

Para isso, vamos utilizar a biblioteca `httr` para fazer requisições HTTP e as bibliotecas `xml2` e `tidyverse` para ler e manipular os dados XML.

```r
if (!"httr" %in% installed.packages()) {
  install.packages("httr")
}
if (!"xml2" %in% installed.packages()) {
  install.packages("xml2")
}
if (!"tidyverse" %in% installed.packages()) {
  install.packages("tidyverse")
}
library(httr)
library(xml2)
library(tidyverse)
```

```
-- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
v dplyr     1.1.4     v readr     2.1.5
v forcats   1.0.0     v stringr   1.5.1
v ggplot2   3.5.2     v tibble    3.3.0
v lubridate 1.9.4     v tidyr     1.3.1
v purrr     1.0.4
-- Conflicts ------------------------------------------ tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()    masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to becom
```

Vamos obter os dados de sessões do mês de abril de 2025:

```
data_inicial <- "01/04/2025"
data_final <- "30/04/2025"
codigo_sessao <- ""
parte_nome_parlamentar <- ""
sigla_partido <- ""
sigla_uf <- ""

url_base <- "https://www.camara.leg.br/sitcamaraws/SessoesReunioes.asmx/ListarDiscursosPlenar

endereco <- str_c(url_base,
                  "?codigoSessao=", codigo_sessao,
                  "&parteNomeParlamentar=", parte_nome_parlamentar,
                  "&siglaPartido=", sigla_partido,
                  "&siglaUF=", sigla_uf,
                  "&dataIni=", data_inicial,
                  "&dataFim=", data_final)

print(endereco)
```

```
[1] "https://www.camara.leg.br/sitcamaraws/SessoesReunioes.asmx/ListarDiscursosPlenario?codig
```

- Como a requisição a ser feita é do tipo GET, os parâmetros são passados na URL. O endereço completo da requisição é construído concatenando a URL base com os parâmetros utilizando a função `str_c` do pacote `stringr`.

```
dados_sessoes <- httr::GET(endereco) |>          # realizar requisição
  httr::content(as = "text") |>                  # obter resposta como texto
  xml2::read_xml() |>                            # ler texto como XML
  xml2::xml_find_all("//sessoesDiscursos") |>    # obter todos os elementos dentro de <sessoes
  xml2::as_list()                                # converter XML para lista

print(length(dados_sessoes))
```

```
[1] 1
```

Os dados incluem informações sobre as sessões, fases das sessões e discursos, incluindo metadados como nome do orador, partido, estado, hora de início do discurso, sumário do discurso etc.

Vamos inspecionar a lista gerada para entender a estrutura:

```r
dados_primeiro_discurso <- dados_sessoes[[1]]$sessao$fasesSessao[[1]]$discursos[[1]]

print(dados_primeiro_discurso)
```

```
$orador
$orador$numero
$orador$numero[[1]]
[1] "1"


$orador$nome
$orador$nome[[1]]
[1] "Benedita da Silva"


$orador$partido
$orador$partido[[1]]
[1] "PT                  "


$orador$uf
$orador$uf[[1]]
[1] "RJ"



$horaInicioDiscurso
$horaInicioDiscurso[[1]]
[1] "30/04/2025 09:20:00"


$txtIndexacao
$txtIndexacao[[1]]
[1] "Deputada federal,Transgênero,Visto consular,Consulado,Estados Unidos,Sexo masculino,Sup


$numeroQuarto
$numeroQuarto[[1]]
[1] "4327688"


$numeroInsercao
```

```
$numeroInsercao[[1]]
[1] "1"


$sumario
$sumario[[1]]
[1] "A Deputada repudiou a emissão de visto com marcação de gênero masculino à Deputada Erika
```

Podemos transformar os dados em data frame (`tibble`) para facilitar a manipulação:

```r
df_sessoes <- dados_sessoes[[1]] |>
  map(as_tibble) |>   # converter cada elemento da lista em tibble
  bind_rows()         # combinar os tibbles em um único tibble

print(df_sessoes)
```

```
# A tibble: 22 x 5
   codigo    data      numero    tipo      fasesSessao
   <list>    <list>    <list>    <list>    <named list>
 1 <chr [1]> <chr [1]> <chr [1]> <chr [1]> <named list [3]>
 2 <chr [1]> <chr [1]> <chr [1]> <chr [1]> <named list [3]>
 3 <chr [1]> <chr [1]> <chr [1]> <chr [1]> <named list [3]>
 4 <chr [1]> <chr [1]> <chr [1]> <chr [1]> <named list [3]>
 5 <chr [1]> <chr [1]> <chr [1]> <chr [1]> <named list [3]>
 6 <chr [1]> <chr [1]> <chr [1]> <chr [1]> <named list [3]>
 7 <chr [1]> <chr [1]> <chr [1]> <chr [1]> <named list [3]>
 8 <chr [1]> <chr [1]> <chr [1]> <chr [1]> <named list [3]>
 9 <chr [1]> <chr [1]> <chr [1]> <chr [1]> <named list [3]>
10 <chr [1]> <chr [1]> <chr [1]> <chr [1]> <named list [3]>
# i 12 more rows
```

Como os dados estão aninhados (sessões > fases > discursos), precisamos expandir os dados em linhas e colunas:

```r
df_sessoes <- df_sessoes |>
  unnest_wider(fasesSessao, names_sep = '_') |>                  # expandir a dados de fases
  unnest(fasesSessao_discursos) |>                               # expandir discursos em lin
  unnest_wider(fasesSessao_discursos, names_sep = '_') |>        # expandir dados de discurs
  unnest_wider(fasesSessao_discursos_orador, names_sep = '_') |> # expandir dados de orador
  mutate_all(compose(str_trim, unlist))                          # remover espaços em branco

print(df_sessoes)
```

```
# A tibble: 1,805 x 15
   codigo  data       numero tipo       fasesSessao_codigo fasesSessao_descricao
   <chr>   <chr>      <chr>  <chr>      <chr>              <chr>
 1 64.2025 30/04/2025 2      Ordinária~ BC                 Breves Comunicações
 2 64.2025 30/04/2025 2      Ordinária~ BC                 Breves Comunicações
 3 64.2025 30/04/2025 2      Ordinária~ BC                 Breves Comunicações
 4 64.2025 30/04/2025 2      Ordinária~ BC                 Breves Comunicações
 5 64.2025 30/04/2025 2      Ordinária~ BC                 Breves Comunicações
 6 64.2025 30/04/2025 2      Ordinária~ BC                 Breves Comunicações
 7 64.2025 30/04/2025 2      Ordinária~ BC                 Breves Comunicações
 8 64.2025 30/04/2025 2      Ordinária~ BC                 Breves Comunicações
 9 64.2025 30/04/2025 2      Ordinária~ BC                 Breves Comunicações
10 64.2025 30/04/2025 2      Ordinária~ BC                 Breves Comunicações
# i 1,795 more rows
# i 9 more variables: fasesSessao_discursos_orador_numero <chr>,
#   fasesSessao_discursos_orador_nome <chr>,
#   fasesSessao_discursos_orador_partido <chr>,
#   fasesSessao_discursos_orador_uf <chr>,
#   fasesSessao_discursos_horaInicioDiscurso <chr>,
#   fasesSessao_discursos_txtIndexacao <chr>, ...
```

Renomeando as colunas para facilitar a leitura:

```r
df_sessoes <- df_sessoes |>
  rename(
    text = fasesSessao_discursos_sumario,
    cod_sessao = fasesSessao_codigo,
    fase_sessao = fasesSessao_descricao,
    num_orador = fasesSessao_discursos_orador_numero,
    nome_orador = fasesSessao_discursos_orador_nome,
    partido_orador = fasesSessao_discursos_orador_partido,
    uf_orador = fasesSessao_discursos_orador_uf,
    hora_inicio_discurso = fasesSessao_discursos_horaInicioDiscurso,
    indexacao = fasesSessao_discursos_txtIndexacao,
    num_quarto = fasesSessao_discursos_numeroQuarto,
    num_insercao = fasesSessao_discursos_numeroInsercao
  ) |>
  mutate(
    doc_id = str_c(cod_sessao, num_quarto, num_orador, num_insercao, sep = "_")
  )

print(df_sessoes)
```

```
# A tibble: 1,805 x 16
   codigo  data       numero tipo  cod_sessao fase_sessao num_orador nome_orador
   <chr>   <chr>      <chr>  <chr> <chr>      <chr>       <chr>      <chr>
 1 64.2025 30/04/2025 2      Ordi~ BC         Breves Com~ 1          Benedita d~
 2 64.2025 30/04/2025 2      Ordi~ BC         Breves Com~ 1          Luiz Lima
 3 64.2025 30/04/2025 2      Ordi~ BC         Breves Com~ 1          Otoni de P~
 4 64.2025 30/04/2025 2      Ordi~ BC         Breves Com~ 1          Sargento F~
 5 64.2025 30/04/2025 2      Ordi~ BC         Breves Com~ 1          Cabo Gilbe~
 6 64.2025 30/04/2025 2      Ordi~ BC         Breves Com~ 1          Helder Sal~
 7 64.2025 30/04/2025 2      Ordi~ BC         Breves Com~ 1          Ricardo Ma~
 8 64.2025 30/04/2025 2      Ordi~ BC         Breves Com~ 1          Rogério Co~
 9 64.2025 30/04/2025 2      Ordi~ BC         Breves Com~ 1          Helder Sal~
10 64.2025 30/04/2025 2      Ordi~ BC         Breves Com~ 1          Coronel As~
# i 1,795 more rows
# i 8 more variables: partido_orador <chr>, uf_orador <chr>,
#   hora_inicio_discurso <chr>, indexacao <chr>, num_quarto <chr>,
#   num_insercao <chr>, text <chr>, doc_id <chr>
```

- o padrão Text Interchange Formats (TIF) foi criado para padronizar objetos de análise textual (https://github.com/ropenscilabs/tif). Para seguir o padrão, armazenamos o texto principal (sumário do discurso) na coluna `text` e um identificador único de cada texto (combinação de sessao, quarto, orador e num_insercao) na coluna `doc_id`.

Algumas estatísticas básicas sobre os discursos:

```r
print(str_c("Número total de discursos: ", nrow(df_sessoes)))
```

```
[1] "Número total de discursos: 1805"
```

```r
print(str_c("Número total de oradores: ", n_distinct(df_sessoes$nome_orador)))
```

```
[1] "Número total de oradores: 346"
```

```r
print(str_c("Número total de partidos: ", n_distinct(df_sessoes$partido_orador)))
```

```
[1] "Número total de partidos: 22"
```

```r
print(str_c("Número total de estados: ", n_distinct(df_sessoes$uf_orador)))
```

```
[1] "Número total de estados: 28"
```

Principais oradores:

```
df_sessoes |>
  group_by(nome_orador) |>           # agrupar por nome do orador
  summarise(num_discursos = n()) |>  # contar número de discursos por orador
  arrange(desc(num_discursos)) |>    # ordenar por número de discursos em ordem decrescente
  head(5)                            # mostrar os 5 oradores com mais discursos
```

```
# A tibble: 5 x 2
  nome_orador         num_discursos
  <chr>                       <int>
1 Cabo Gilberto Silva            57
2 Luiz Lima                      48
3 Hildo Rocha                    39
4 Marcel van Hattem              39
5 Bibo Nunes                     36
```

Principais partidos:

```
df_sessoes |>
  group_by(partido_orador) |>        # agrupar por nome do partido
  summarise(num_discursos = n()) |>  # contar número de discursos por orador
  arrange(desc(num_discursos)) |>    # ordenar por número de discursos em ordem decrescente
  head(5)                            # mostrar os 5 oradores com mais discursos
```

```
# A tibble: 5 x 2
  partido_orador num_discursos
  <chr>                  <int>
1 PL                       448
2 PT                       362
3 PSOL                     129
4 UNIÃO                    117
5 MDB                      107
```

Para facilitar novos processamentos, podemos salvar os dados em formato CSV:

```
write_csv(df_sessoes, "data/dados_sessoes.csv")
```

Da mesma forma, podemos ler arquivos CSV utilizando a função `read_csv` do pacote `readr`, incluído na coleção `tidyverse`:

```r
df_sessoes <- read_csv("data/dados_sessoes.csv")
```

```
Rows: 1805 Columns: 16
-- Column specification --------------------------------------------------------
Delimiter: ","
chr (12): codigo, data, tipo, cod_sessao, fase_sessao, nome_orador, partido_...
dbl  (4): numero, num_orador, num_quarto, num_insercao

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

2.1.2 Criando objeto corpus

Corpus é um termo utilizado para se referir a um conjunto de textos que serão analisados.

O corpus é a base para muitas análises de PLN, pois é a partir dele que os modelos aprendem padrões e estruturas da linguagem.

O termo *corpora* se refere à uma coleção de corpus.

A biblioteca `quanteda` possui diversas funções para criar e manipular corpora, como `corpus()`:

```r
if (!"quanteda" %in% installed.packages()) {
  install.packages("quanteda")
}
library(quanteda)
```

```
Package version: 4.3.1
Unicode version: 15.0
ICU version: 72.1

Parallel computing: 12 of 12 threads used.

See https://quanteda.io for tutorials and examples.
```

Por padrão, o corpus é criado utilizando a coluna `text` como texto principal e a coluna `doc_id` como identificador único de cada texto:

```r
corpus_sessoes <- corpus(df_sessoes, text_field = "text", docid_field = "doc_id")

print(corpus_sessoes)
```

```
Corpus consisting of 1,805 documents and 14 docvars.
BC_4327688_1_1 :
"A Deputada repudiou a emissão de visto com marcação de gêner..."

BC_4327698_1_1 :
"O Deputado criticou o Governo Lula por abandonar os mais pob..."

BC_4327701_1_1 :
"O Deputado acusou a família do Presidente Lula de se benefic..."

BC_4327704_1_1 :
"O Deputado alertou para a tentativa de um líder da organizaç..."

BC_4327707_1_1 :
"O Deputado criticou o Ministro da Previdência Social, Carlos..."

BC_4327710_1_1 :
"O Deputado defendeu o papel do Governo Lula na investigação ..."

[ reached max_ndoc ... 1,799 more documents ]
```

Os metadados (informações adicionais sobre os textos) são armazenados como atributos do corpus e podem ser acessados utilizando a função `docvars()`:

```r
metadados <- corpus_sessoes |>
  docvars() |>                                  # obter metadados do corpus
  select(nome_orador, partido_orador, uf_orador) # selecionar colunas de interesse

print(head(metadados)) # head mostra as primeiras linhas do data frame
```

```
          nome_orador partido_orador uf_orador
1   Benedita da Silva             PT        RJ
2           Luiz Lima           NOVO        RJ
3      Otoni de Paula            MDB        RJ
4      Sargento Fahur            PSD        PR
5 Cabo Gilberto Silva             PL        PB
6      Helder Salomão             PT        ES
```

Assim como `filter`, podemos obter subconjuntos de `corpus_sessoes` com a função `corpus_subset()`:

```
corpus_sessoes_pt <- corpus_subset(corpus_sessoes, partido_orador == "PT")

print(corpus_sessoes_pt)
```

```
Corpus consisting of 362 documents and 14 docvars.
BC_4327688_1_1 :
"A Deputada repudiou a emissão de visto com marcação de gêner..."

BC_4327710_1_1 :
"O Deputado defendeu o papel do Governo Lula na investigação ..."

BC_4327717_1_1 :
"O Deputado condenou o uso da imunidade parlamentar como escu..."

BC_4327721_1_1 :
"O Deputado anunciou a realização da 26ª Marcha pela Vida e C..."

BC_4327728_1_1 :
"O Deputado criticou a desonestidade intelectual de Parlament..."

BC_4327734_1_1 :
"O Deputado saudou a presença na Câmara dos Deputados de Marc..."

[ reached max_ndoc ... 356 more documents ]
```

A biblioteca `quanteda.textstats` possui diversas funções para calcular estatísticas de texto, como frequência de palavras, diversidade lexical, legibilidade etc.

```
if (!"quanteda.textstats" %in% installed.packages()) {
  install.packages("quanteda.textstats")
}
library(quanteda.textstats)
```

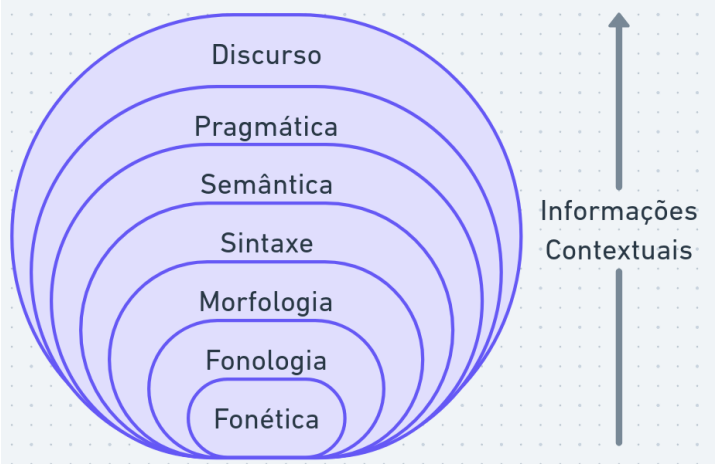
Da mesma forma, `quatenda.textplots` possui diversas funções para visualização de texto, como nuvens de palavras, gráficos de frequência, gráficos de coocorrência etc.

```r
if (!"quanteda.textplots" %in% installed.packages()) {
  install.packages("quanteda.textplots")
}
library(quanteda.textplots)
```

A seguir, vamos utilizar `corpus_sessoes`, as bibliotecas `quanteda` e relacionadas para realizar as análises de palavras.

2.2 Palavras, morfemas, lemas e stems

O estudo da língua é dividido em diversas subáreas:



- **fonética** e **fonologia**: sons e sua organização
- **morfologia**: como mofemas se organizam para formar palavras
- **sintaxe**: como palavras se organizam para formar sintagmas e orações
- **semântica**: significado de palavras e frases
- **pragmática**: organização de orações para fins comunicativos
- **discurso**: estruturas do texto como todo

O estudo de PLN geralmente tem foco no *texto escrito* (e.g morfologia, sintaxe, semântica, discurso).

Porém, fonética e fonologia também são importantes para PLN, especialmente em tarefas que envolvem *fala* (e.g. reconhecimento de fala, síntese de fala).

Além disso, dada a natureza multimodal da análise de discurso a fala também é relevante. Por exemplo:

https://www.camara.leg.br/internet/sitaqweb/TextoHTML.asp?etapa=5&nuSessao=64.2025&nuQuarto=4327701&nuOrador=1&nuInsercao=1&dtHorarioQuarto=09:32&sgFaseSessao=BC&Data=30/04/2025&txApelido=Otoni%20de%20Paula&txFaseSessao=Breves%20Comunica%C3%A7%C3%B5es&txTipoSessao=Ordin%C3%A1ria%20-%20CD&dtHoraQuarto=09:32&txEtapa=

> Que tipo de informação é transmitida por meio da fala, além do conteúdo semântico? Quais estão destacadas no texto taquigrafado?

A unidade mínima de informação para PLN depende do nível de análise:

- partes de palavras (como morfemas, lemas ou stems) são uteis para "unir" signicados entre palavras relacionadas
- palavras são uteis para análise sintática e semântica
- frases e parágrafos são uteis para análise de modelos de discurso

Por exemplo "beija-flor" e "escova de dente" podem ser consideradas palavras compostas ou não.

Na morfologia, a unidade mínima de significado é o **morfema**:

- "casa" tem um morfema (casa)
- "casas" tem dois morfemas (casa + s)
- "casinha" tem dois morfemas (casa + inha)
- "casinhas" tem três morfemas (casa + inha + s)

A **lematização** é o processo de reduzir uma palavra ao seu lema (ou forma canônica). Ex.:

- "gatos" -> "gato"
- "gata" -> "gato"
- "gatinhas" -> "gato"

A **stemização** é o processo (menos preciso) de reduzir uma palavra ao seu radical (ou raiz). Ex.:

- "gatos" -> "gat"
- "gata" -> "gat"
- "gatinhas" -> "gat"

2.2.1 Obtendo informações morfológicas de palavras

Separar palavras em subunidades pode ser útil em análises:

- dividir palavras compostas
- tratar palavras novas ou desconhecidas
- analisar palavras incorretas
- avaliar complexidade do texto

Por exemplo, Moffitt (2016) destaca o uso de neologismos em performances populistas (*refudiate* por Sarah Palin). Outros estudos comparam morfemas em diferentes estilos de texto, como jornalístico e literário.

A biblioteca `morphemepiece` é uma ferramenta para segmentação de palavras em morfemas da língua inglesa.

```
if (!"morphemepiece" %in% installed.packages()) {
  install.packages("morphemepiece")
}
library(morphemepiece)

print(morphemepiece_tokenize("houses"))
```

```
[[1]]
house   ##s
 5617  4034
```

```
print(morphemepiece_tokenize("orderly"))
```

```
[[1]]
order  ##ly
 6478  4057
```

```
print(morphemepiece_tokenize("paradoxical"))
```

```
[[1]]
paradox  ##ical
   9590    4361
```

```r
print(morphemepiece_tokenize("discoursal"))
```

```
[[1]]
discourse      ##al
     6456      4041
```

Existem ferramentas baseadas em dicionários e língua portuguesa do Brasil, por exemplo o projeto MorphoBR https://github.com/LR-POR/MorphoBr

```
echo "fortinho" | flookup -i morphobr.bin
fortinho    forte+N+DIM+M+SG
fortinho    forte+A+DIM+M+SG
```

- lema: forte
- N: nome
- A: adjetivo
- DIM: diminutivo
- M: masculino
- SG: singular

2.2.2 Stemming

Stemming é um processo mais simples e menos preciso do que lematização, mas pode ser útil para algumas tarefas de PLN, como análise de sentimentos ou classificação de texto.

Exemplos utilizando a biblioteca `quanteda`:

```r
stems_testar <- char_wordstem(c("testar", "testando", "testarei"), language = "pt")

print(stems_testar)
```

```
[1] "test" "test" "test"
```

```r
stems_love <- char_wordstem(c("love", "loving", "loved"), language = "en")

print(stems_love)
```

```
[1] "love" "love" "love"
```

Stemming vai reduzir o vocabulário do texto, facilitando algumas análises de frequência, mas pode gerar ambiguidades e perder informações importantes sobre o significado das palavras.

2.2.3 Caracteres

A menor unidade de informação em um texto *computacional* (ou string) é o **caractere** (ou símbolo).

O padrão *Unicode* cataloga e codifica caracteres de diferentes linguagens, incluindo linguagens extintas como sumério e cuneiforme:

Ver: <>

A **codificação (encoding)** de textos se refere a como os caracteres são representados em números binários no computador (bytes).

O padrão mais comum é o **UTF-8** que utiliza tamanho variável de caracteres.

A biblioteca `quanteda` possui funções para separação de caracteres, como `tokenize_character`:

```
print(tokenize_character("avião =  "))
```

```
[[1]]
 [1] "a"  "v"  "i"  "ã"  "o"  " "  "="  " "  " " " "
```

```
print(tokenize_character("#feliz! \U0001f600"))
```

```
[[1]]
[1] "#"  "f"  "e"  "l"  "i"  "z"  "!"  " "  " "
```

2.2.4 Tokenização

Não há consenso sobre a definição de *palavra* na linguística. Em PLN, a separação do texto em palavras depende do contexto da aplicação.

O problema de separar palavras (ou unidades mínimas de significado) em um texto é conhecido como **tokenização** e é uma etapa fundamental para muitas tarefas de PLN.

Note que o problema não é muito simples. O chinês, por exemplo, não utiliza espaço para separar palavras e cada caractere representa um morfema.

A biblioteca `quanteda` possui a função `tokens()` para realizar a tokenização de um texto, com diversas opções para personalizar o processo:

```r
texto <- 'Ele disse: "Uh, são 13 horas. Não está na hora de almoçar?"'

tokens_padrao <- tokens(texto)

print(tokens_padrao)
```

```
Tokens consisting of 1 document.
text1 :
 [1] "Ele"   "disse" ":"     "\""    "Uh"    ","     "são"   "13"    "horas"
[10] "."     "Não"   "está"
[ ... and 6 more ]
```

```r
texto <- 'Ele disse: "Uh, são 13 horas. Não está na hora de almoçar?"'

tokens <- tokens(texto, remove_punct = TRUE, remove_numbers = TRUE)

print(tokens)
```

```
Tokens consisting of 1 document.
text1 :
 [1] "Ele"     "disse"   "Uh"      "são"     "horas"   "Não"     "está"
 [8] "na"      "hora"    "de"      "almoçar"
```

```r
texto <- 'Ele disse: "Uh, são 13 horas. Não está na hora de almoçar?". Não, ela disse.'

tokens <- tokens(texto, what = "sentence")

print(tokens)
```

```
Tokens consisting of 1 document.
text1 :
[1] "Ele disse: \"Uh, são 13 horas."  "Não está na hora de almoçar?\"."
[3] "Não, ela disse."
```

A tokenização em palavras é útil para análise sintática e semântica, enquanto a tokenização em sentenças é útil para análise de modelos de discurso.

Em algumas aplicações, como modelos de linguagem, a tokenização é um processo essencial na contrução do **vocabulário**. O vocabulário é a coleção de todas as palavras conhecidas do modelo.

Uma estrutura útil npara análise de texto é a **matriz de documentos e termos (Document-Feature Matrix, DFM)**, que representa a frequência de cada palavra em cada documento:

```
dfm_sessoes <- corpus_sessoes |>
  tokens(remove_punct = TRUE, remove_numbers = TRUE) |> # tokenizar texto
  tokens_tolower() |>                                   # converter palavras para minúsculo
  dfm()                                                 # criar matriz de documentos e termos

print(dfm_sessoes)
```

```
Document-feature matrix of: 1,805 documents, 11,087 features (99.40% sparse) and 14 docvars.
                features
docs             a deputada repudiou emissão de visto com marcação gênero
  BC_4327688_1_1 5        2        1       1  7     1   3        1      3
  BC_4327698_1_1 3        0        1       0  6     0   0        0      0
  BC_4327701_1_1 3        0        0       0  6     0   0        0      0
  BC_4327704_1_1 7        0        0       0 12     0   1        0      0
  BC_4327707_1_1 3        0        0       0  5     0   1        0      0
  BC_4327710_1_1 4        0        0       0  7     0   4        0      0
                features
docs             masculino
  BC_4327688_1_1         1
  BC_4327698_1_1         0
  BC_4327701_1_1         0
  BC_4327704_1_1         0
  BC_4327707_1_1         0
  BC_4327710_1_1         0
[ reached max_ndoc ... 1,799 more documents, reached max_nfeat ... 11,077 more features ]
```

Cada linha é um documento e cada coluna é uma palavra. O valor em cada célula representa a frequência da palavra no documento.

```
print(dfm_sessoes[3, "presidente"])
```

```
Document-feature matrix of: 1 document, 1 feature (0.00% sparse) and 14 docvars.
                features
docs             presidente
  BC_4327701_1_1          1
```

Por exemplo, para contar quantas vezes a palavra "presidente" é mencionada em todos os discursos, podemos somar os valores da coluna correspondente:

```r
sum(dfm_sessoes[, "presidente"])
```

```
[1] 341
```

2.2.5 Byte Pair Encoding (BPE)

A divisão em simples palavras tem um problema com palavras desconhecidas (*out-of-vocabulary*).

Uma solução que busca o "meio-termo" entre a separação entre palavras e a separação entre caracteres é o **Byte Pair Encoding (BPE)**.

O BPE é um algoritmo de compressão de dados que pode ser adaptado para tokenização. Ele funciona substituindo pares caracteres mais frequentes por um token único, recursivamente, criando assim um vocabulário de subpalavras.

```r
if (!"tokenizers.bpe" %in% installed.packages()) {
  install.packages("tokenizers.bpe")
}
library(tokenizers.bpe)
```

```r
model <- bpe(
  df_sessoes$text,
  vocab_size = 200000,
  threads = 1,
  model_path = "data/bpe_tokenizer"
)
```

```r
bpe_encode(model, c("adultização", "adulto", "eleitoreado", "eleitor", "sexta-feira", "sexto
```

```
[[1]]
[1] " adul"   "tização"

[[2]]
[1] " adul" "to"

[[3]]
[1] " eleito" "re"      "ado"

[[4]]
[1] " eleito" "r"
```

```
[[5]]
[1] " sex" "ta-f" "eira"

[[6]]
[1] " sex" "tou"
```

Note que os tokens gerados pelo BPE não necessáriamente correspontem "significados".

Porém, são muito útens em modelos que consideram sequências como entrada e modelos multi-línguas.

2.2.6 Stopwords

Stopwords são palavras comuns que geralmente não carregam muito significado semântico, como "o", "a", "de", "e" em português. Elas são frequentemente removidas em tarefas de PLN para reduzir o ruído e melhorar a eficiência do modelo.

Uma prática comum no processamento de textos é remover as stopwords antes de realizar análises, como contagem de palavras ou análise de sentimentos.

Podemos encontrar listas de stopwords em português e outras línguas na Internet, por exemplo, utilizando o pacote `stopwords`.

```r
if (!"stopwords" %in% installed.packages()) {
  install.packages("stopwords")
}
library(stopwords)
```

```r
stopwords_portugues <- stopwords(language = "pt")

print(stopwords_portugues[1:10])
```

```
 [1] "de"   "a"    "o"    "que"  "e"    "do"   "da"   "em"   "um"   "para"
```

A remoção de stopwords pode não ser adequada para todas as tarefas, especialmente aquelas que dependem do contexto ou da estrutura do texto (análise sintática, partes de discurso, etc).

2.3 Exemplo: Palavras mais Utilizadas em Discursos

Primeiro, vamos tokenizar os discursos utilizando a função `spacy_tokenize` e expandir os tokens em linhas utilizando a função `unnest` do pacote `tidyverse`:

```
tokens_sessoes <- corpus_sessoes |>
  tokens(remove_punct = TRUE, remove_symbols = TRUE, remove_numbers = TRUE)

print(tokens_sessoes)
```

```
Tokens consisting of 1,805 documents and 14 docvars.
BC_4327688_1_1 :
 [1] "A"         "Deputada"  "repudiou"  "a"         "emissão"   "de"
 [7] "visto"     "com"       "marcação"  "de"        "gênero"    "masculino"
[ ... and 124 more ]

BC_4327698_1_1 :
 [1] "O"         "Deputado"  "criticou"  "o"         "Governo"   "Lula"
 [7] "por"       "abandonar" "os"        "mais"      "pobres"    "e"
[ ... and 117 more ]

BC_4327701_1_1 :
 [1] "O"              "Deputado"       "acusou"         "a"
 [5] "família"        "do"             "Presidente"     "Lula"
 [9] "de"             "se"             "beneficiar"     "reiteradamente"
[ ... and 107 more ]

BC_4327704_1_1 :
 [1] "O"           "Deputado"    "alertou"     "para"        "a"
 [6] "tentativa"   "de"          "um"          "líder"       "da"
[11] "organização" "criminosa"
[ ... and 153 more ]

BC_4327707_1_1 :
 [1] "O"           "Deputado"    "criticou"    "o"           "Ministro"
 [6] "da"          "Previdência" "Social"      "Carlos"      "Lupi"
[11] "por"         "não"
[ ... and 104 more ]

BC_4327710_1_1 :
 [1] "O"            "Deputado"     "defendeu"     "o"            "papel"
 [6] "do"           "Governo"      "Lula"         "na"           "investigação"
```

```
[11] "e"            "no"
[ ... and 110 more ]

[ reached max_ndoc ... 1,799 more documents ]
```

Agora vamos transformar todas as palavras em minúsculo e remover stopwords utilizando a função `tokens_remove`:

```r
stopwords_portugues <- stopwords(language = "pt")

tokens_sessoes <- tokens_sessoes |>
  tokens_tolower() |>
  tokens_remove(stopwords_portugues)

print(tokens_sessoes)
```

```
Tokens consisting of 1,805 documents and 14 docvars.
BC_4327688_1_1 :
 [1] "deputada"  "repudiou"  "emissão"   "visto"     "marcação"  "gênero"
 [7] "masculino" "deputada"  "erika"     "hilton"    "consulado" "estados"
[ ... and 69 more ]

BC_4327698_1_1 :
 [1] "deputado"  "criticou"  "governo"   "lula"      "abandonar" "pobres"
 [7] "repudiou"  "escândalo" "fraudes"   "instituto" "nacional"  "seguro"
[ ... and 68 more ]

BC_4327701_1_1 :
 [1] "deputado"       "acusou"         "família"        "presidente"
 [5] "lula"           "beneficiar"     "reiteradamente" "recursos"
 [9] "públicos"       "apontou"        "reportagem"     "sobre"
[ ... and 64 more ]

BC_4327704_1_1 :
 [1] "deputado"    "alertou"     "tentativa"   "líder"       "organização"
 [6] "criminosa"   "primeiro"    "comando"     "capital"     "pcc"
[11] "rodrigo"     "felício"
[ ... and 97 more ]

BC_4327707_1_1 :
 [1] "deputado"        "criticou"        "ministro"        "previdência"
 [5] "social"          "carlos"          "lupi"            "prestar"
```

```
 [9] "esclarecimentos" "sobre"           "desvio"          "r"
[ ... and 58 more ]

BC_4327710_1_1 :
 [1] "deputado"        "defendeu"        "papel"           "governo"
 [5] "lula"            "investigação"    "desmantelamento" "esquema"
 [9] "fraudulento"     "desviava"        "recursos"        "aposentados"
[ ... and 53 more ]

[ reached max_ndoc ... 1,799 more documents ]
```

Agora vamos construir a DFM:

```r
dfm_sessoes <- dfm(tokens_sessoes)

print(dfm_sessoes)
```

```
Document-feature matrix of: 1,805 documents, 10,962 features (99.55% sparse) and 14 docvars.
                features
docs             deputada repudiou emissão visto marcação gênero masculino
  BC_4327688_1_1        2        1       1     1        1      3         1
  BC_4327698_1_1        0        1       0     0        0      0         0
  BC_4327701_1_1        0        0       0     0        0      0         0
  BC_4327704_1_1        0        0       0     0        0      0         0
  BC_4327707_1_1        0        0       0     0        0      0         0
  BC_4327710_1_1        0        0       0     0        0      0         0
                features
docs             erika hilton consulado
  BC_4327688_1_1     1      1         1
  BC_4327698_1_1     0      0         0
  BC_4327701_1_1     0      0         0
  BC_4327704_1_1     0      0         0
  BC_4327707_1_1     0      0         0
  BC_4327710_1_1     0      0         0
[ reached max_ndoc ... 1,799 more documents, reached max_nfeat ... 10,952 more features ]
```

Para extrair algumas estatísticas básicas, como as palavras mais comuns, podemos utilizar a função `topfeatures`:

```r
palavras_mais_comuns <- textstat_frequency(dfm_sessoes, n = 10)

print(palavras_mais_comuns)
```

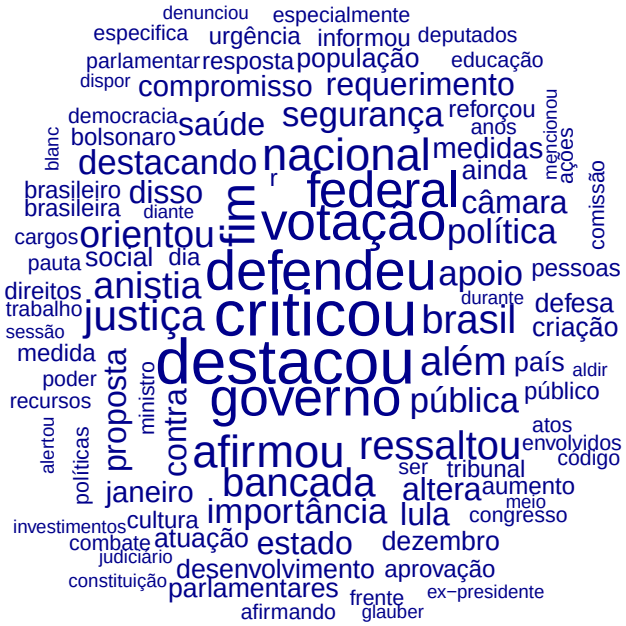
```
    feature frequency rank docfreq group
1  deputado      1682    1    1487   all
2       lei      1127    2     677   all
3        nº      1113    3     695   all
4   projeto       830    4     740   all
5     sobre       664    5     541   all
6  criticou       644    6     603   all
7  destacou       630    7     617   all
8  defendeu       542    8     516   all
9   governo       532    9     388   all
10  votação       508   10     459   all
```

Podemos remover mais palavras que não colaboram para a interpretação dos dados, da mesma forma que removemos stopwords:

```r
palavras_irrelevantes <- c("senhor", "senhora", "sobre", "presidente", "deputado", "deputada

dfm_sessoes <- dfm_remove(dfm_sessoes, palavras_irrelevantes)

palavras_mais_comuns <- textstat_frequency(dfm_sessoes, n = 10)

print(palavras_mais_comuns)
```

```
    feature frequency rank docfreq group
1  criticou       644    1     603   all
2  destacou       630    2     617   all
3  defendeu       542    3     516   all
4   governo       532    4     388   all
5   votação       508    5     459   all
6   federal       498    6     409   all
7       fim       445    7     435   all
8   afirmou       438    8     408   all
9  nacional       435    9     390   all
10  justiça       403   10     337   all
```

```r
textplot_wordcloud(dfm_sessoes, max_words = 100)
```



Agrupando por partido:

```
palavras_por_partido <- textstat_frequency(dfm_sessoes, n = 5, groups = partido_orador)

print(palavras_por_partido)
```

```
        feature frequency rank docfreq         group
1       queiroz        38    1      10            --
2      destacou        36    2      35            --
3     discursou        36    2      36            --
4        sessão        35    4      34            --
5        solene        35    4      34            --
6         apoio         8    1       5        AVANTE
7      destacou         7    2       7        AVANTE
8         bahia         7    2       5        AVANTE
9       votação         7    2       6        AVANTE
10       brasil         6    5       6        AVANTE
11      autismo         3    1       2     CIDADANIA
12       pessoa         2    2       1     CIDADANIA
13     defendeu         2    2       2     CIDADANIA
14     respeito         2    2       2     CIDADANIA
15     direitos         2    2       2     CIDADANIA
16     destacou        44    1      42           MDB
```

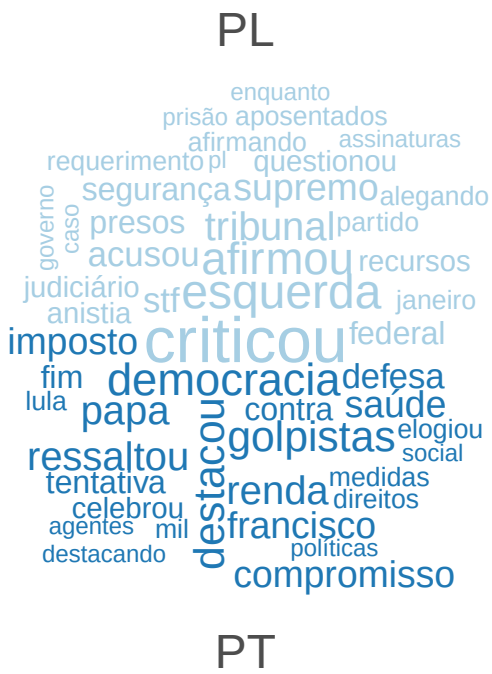
17	votação	29	2	27	MDB
18	criticou	28	3	25	MDB
19	defendeu	26	4	24	MDB
20	bancada	24	5	24	MDB
21	votação	54	1	50	NOVO
22	criticou	36	2	31	NOVO
23	governo	32	3	21	NOVO
24	federal	29	4	20	NOVO
25	bancada	29	4	28	NOVO
26	destacou	26	1	26	PCdoB
27	importância	21	2	20	PCdoB
28	defendeu	19	3	17	PCdoB
29	contra	18	4	16	PCdoB
30	nacional	16	5	15	PCdoB
31	votação	17	1	15	PDT
32	bancada	15	2	15	PDT
33	destacou	13	3	13	PDT
34	orientou	13	3	13	PDT
35	justiça	12	5	12	PDT
36	criticou	231	1	217	PL
37	governo	182	2	126	PL
38	federal	158	3	131	PL
39	afirmou	156	4	145	PL
40	votação	140	5	121	PL
41	criticou	17	1	16	PODE
42	destacou	15	2	15	PODE
43	defendeu	13	3	12	PODE
44	governo	13	3	10	PODE
45	nacional	13	3	12	PODE
46	jornal	3	1	1	PODEMOS
47	estado	2	2	1	PODEMOS
48	estados	1	3	1	PODEMOS
49	unidos	1	3	1	PODEMOS
50	contrário	1	3	1	PODEMOS
51	destacou	26	1	25	PP
52	apoio	17	2	15	PP
53	importância	17	2	17	PP
54	destacando	16	4	14	PP
55	estado	16	4	14	PP
56	destacou	4	1	4	PRD
57	descontos	4	1	2	PRD
58	pública	4	1	3	PRD
59	valorização	4	1	3	PRD

60	proposta	4	1	3	PRD
61	votação	24	1	23	PSB
62	bancada	23	2	23	PSB
63	orientou	21	3	21	PSB
64	justiça	17	4	7	PSB
65	destacou	15	5	13	PSB
66	destacou	45	1	45	PSD
67	criticou	36	2	31	PSD
68	defendeu	32	3	30	PSD
69	fim	31	4	30	PSD
70	federal	30	5	25	PSD
71	federal	9	1	5	PSDB
72	além	9	1	7	PSDB
73	destacou	8	3	8	PSDB
74	defendeu	8	3	7	PSDB
75	social	6	5	5	PSDB
76	votação	53	1	48	PSOL
77	criticou	49	2	48	PSOL
78	glauber	43	3	26	PSOL
79	contra	39	4	33	PSOL
80	defendeu	33	5	31	PSOL
81	destacou	134	1	132	PT
82	governo	133	2	101	PT
83	defendeu	129	3	123	PT
84	criticou	123	4	116	PT
85	fim	107	5	104	PT
86	federal	10	1	6	PV
87	destacou	9	2	7	PV
88	governo	8	3	5	PV
89	defendeu	7	4	6	PV
90	nacional	7	4	5	PV
91	bancada	2	1	2	REDE
92	nacional	2	1	2	REDE
93	aldir	2	1	2	REDE
94	blanc	2	1	2	REDE
95	política	2	1	2	REDE
96	destacou	51	1	50	REPUBLICANOS
97	ressaltou	34	2	31	REPUBLICANOS
98	defendeu	34	2	33	REPUBLICANOS
99	nacional	27	4	24	REPUBLICANOS
100	fim	26	5	26	REPUBLICANOS
101	destacou	6	1	5	SOLIDARIEDADE
102	nacional	5	2	4	SOLIDARIEDADE

```
103          mg         5    2       3 SOLIDARIEDADE
104 importância         4    4       4 SOLIDARIEDADE
105   prevenção         4    4       1 SOLIDARIEDADE
106    criticou        51    1      46         UNIÃO
107    defendeu        50    2      48         UNIÃO
108     governo        48    3      35         UNIÃO
109    destacou        46    4      46         UNIÃO
110   segurança        42    5      27         UNIÃO
```

Para comparar as palavras mais comuns entre partidos, podemos utilizar a função `textplot_wordcloud` com o argumento `comparison = TRUE`:

```
dfm_sessoes |>
  dfm_subset(partido_orador %in% c("PT", "PL")) |>       # filtrar partidos PT e PL
  dfm_group(groups = partido_orador) |>                  # agrupar por partido
  textplot_wordcloud(comparison = TRUE, max_words = 50)  # gerar wordcloud comparativo
```



2.4 Para saber mais

Explore a documentação da API de dados abertos da Câmara dos Deputados: https://www2.camara.leg.br/transparencia/dados-abertos/dados-abertos-legislativo/webservices/sessoesreunioes-2.

Leia mais sobre morfemas, tokenização no Capítulo 2 do livro "Speech and Language Processing" (Jurafsky and Martin 2026) e os Capítulos 4 e 5 do livro "Processamento de Linguagem Natural" (Caseli and Nunes 2024).

Estude os tutoriais da biblioteca `quanteda` em https://tutorials.quanteda.io/. Saiba mais sobre a biblioteca lendo o artigo de Benoit et al. (2018).

2.5 Atividade prática

1. Obtenha os discursos do plenário da Câmara dos Deputados utilizando a API para um período de sua escolha.
2. Realize a tokenização e remova stopwords dos discursos.
3. Conte as palavras mais comuns e visualize os resultados utilizando uma nuvem de palavras (word cloud).
4. Compare as palavras mais comuns entre diferentes partidos ou oradores.
5. Utilize o argumento `groups` para construir o `dfm(..., groups = "partido_orador")` e `textplot_wordcloud(..., comparison = TRUE)` para gerar wordclouds comparativos.

Glossário

Introdução

ADP Análise de Discurso Político
PLN (NLP) Processamento de Linguagem Natural (Natural Language Processing)

Aula 1

R linguagem de programação utilizada para análise de dados
RStudio ambiente de desenvolvimento integrado (IDE) para R
Tidyverse coleção de pacotes R para ciência de dados
Variável nome para um valor armazenado em memória, que pode ser alterado
Condicional estrutura de código que executa ou não um bloco, de acordo com uma expressão lógica
Função um trecho de código que possui um nome, geralmente para uso de operações comuns
|> operador pipe utilizado para encadear chamadas de funções (o resultado é passado como argumento para a próxima função)
Vetor coleção ou sequência de um ou mais valores do mesmo tipo
Data Frame estrutura de dado para armazenar tabelas com linhas e colunas
Biblioteca coleção de funções, pode ser instalada e compartilhada em repositórios na Internet
String cadeia de caracteres, tipo de dado utilizado para representar texto

Referências

Benoit, Kenneth, Kohei Watanabe, Haiyan Wang, Paul Nulty, Adam Obeng, Stefan Müller, and Akitaka Matsuo. 2018. "Quanteda: An r Package for the Quantitative Analysis of Textual Data." *Journal of Open Source Software* 3 (30): 774. https://doi.org/10.21105/joss.00774.

Caseli, H. M., and M. G. V. Nunes, eds. 2024. *Processamento de Linguagem Natural: Conceitos, Técnicas e Aplicações Em Português*. 2nd ed. BPLN. https://brasileiraspln.com/livro-pln/2a-edicao/.

Jurafsky, Daniel, and James H. Martin. 2026. *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition, with Language Models*. 3rd ed. https://web.stanford.edu/~jurafsky/slp3/.

Moffitt, Benjamin. 2016. *The Global Rise of Populism: Performance, Political Style, and Representation*. 1st ed. Stanford University Press. https://doi.org/10.2307/j.ctvqsdsd8.

O'Neill, Lachlan, Nandini Anantharama, Wray Buntine, and Simon D. Angus. 2021. "Quantitative Discourse Analysis at Scale - AI, NLP and the Transformer Revolution." *SoDa Laboratories Working Paper Series*, SoDa Laboratories Working Paper Series, December. https://ideas.repec.org//p/ajr/sodwps/2021-12.html.

Silge, Julia, and David Robinson. 2017. *Text mining with R: a tidy approach*. Beijing, China: O'Reilly.

Wickham, Hadley. 2023. *R for Data Science*. 2nd ed. Sebastopol: O'Reilly Media, Incorporated.

Expressões regulares

- Expressões regulares são padrões utilizados para buscar e manipular texto
- Permitem encontrar palavras, frases ou padrões específicos em um texto

A biblioteca `stringr` do R é uma das mais utilizadas para trabalhar com expressões regulares.

```
if (!"stringr" %in% installed.packages()) {
  install.packages("stringr")
}
library(stringr)
```

A função `str_detect()` é utilizada para verificar se um padrão específico está presente em uma string.

Exemplo de expressão regular para encontrar textos que contém qualquer palavra iniciando com "brasil":

```
discursos <- c(
  "o Brasil é um país maravilhoso",
  "a economia do Japão está crescendo",
  "o futebol argentino é famoso",
  "brasileiras são conhecidas por sua alegria",
  "o clima no Brasil é tropical",
  "brasileiros são apaixonados por futebol"
)

contem_brasil <- str_detect(discursos, "brasil.*")
print(discursos[contem_brasil])
```

```
[1] "brasileiras são conhecidas por sua alegria"
[2] "brasileiros são apaixonados por futebol"
```

- o ponto `.` significa "qualquer caractere" e o asterisco `*` significa "zero ou mais vezes". Portanto, a expressão regular `brasil.*` corresponde a qualquer string que contenha a palavra "brasil" seguida de zero ou mais caracteres, ou seja, qualquer string que contenha a palavra "brasil" em qualquer posição.

Para encontrar o padrão "Brasil" (com B maiúsculo) ou "brasil" podemos utilizar:

```
contem_brasil <- str_detect(discursos, "brasil.*|Brasil.*")
print(discursos[contem_brasil])
```

```
[1] "o Brasil é um país maravilhoso"
[2] "brasileiras são conhecidas por sua alegria"
[3] "o clima no Brasil é tropical"
[4] "brasileiros são apaixonados por futebol"
```

`<re1>|<re2>` vai detectar a expressão `re1` ou `re2`. Ou, de forma mais concisa:

```
contem_brasil <- str_detect(discursos, "[Bb]rasil.*")
print(discursos[contem_brasil])
```

```
[1] "o Brasil é um país maravilhoso"
[2] "brasileiras são conhecidas por sua alegria"
[3] "o clima no Brasil é tropical"
[4] "brasileiros são apaixonados por futebol"
```

- Os colchetes `[]` indicam um conjunto de caracteres. Portanto, a expressão regular `[Bb]rasil.*` corresponde a qualquer string que contenha "Brasil" com B maiúsculo ou "brasil" com b minúsculo, seguida de zero ou mais caracteres.

Podemos procurar por discursos que começam com "brasil" ou "Brasil" utilizando o símbolo `^` para indicar o início da string:

```
discursos <- c(
  "Brasil é um país maravilhoso",
  "a economia do Japão está crescendo",
  "o futebol argentino é famoso",
  "brasileiras são conhecidas por sua alegria",
  "o clima no Brasil é tropical",
  "brasileiros são apaixonados por futebol"
)

começam_com_brasil <- str_detect(discursos, "^[bB]rasil")
print(discursos[começam_com_brasil])
```

```
[1] "Brasil é um país maravilhoso"
[2] "brasileiras são conhecidas por sua alegria"
[3] "brasileiros são apaixonados por futebol"
```

De forma análioga o símbolo ` indica o final da string. Para encontrar discursos que terminam com "o":

```
discursos <- c(
  "Brasil é um país maravilhoso",
  "a economia do Japão está crescendo",
  "o futebol argentino é famoso",
  "brasileiras são conhecidas por sua alegria",
  "o clima no Brasil é tropical",
  "brasileiros são apaixonados por futebol"
)

começam_com_brasil <- str_detect(discursos, "o$")
print(discursos[começam_com_brasil])
```

```
[1] "Brasil é um país maravilhoso"       "a economia do Japão está crescendo"
[3] "o futebol argentino é famoso"
```